R felfedező munkalap - MEGOLDÓKULCS

Tanári változat: rövid magyarázat, fontos eredmény és a tanulság minden kifejezéshez.

1. Vektorok létrehozása a c() függvénnyel

1. `c(1,2,3,4,5,6)`

Magyarázat:
Számokból álló numerikus vektort hoz létre. A c() jelentése combine/concatenate: az elemeket egy vektorba fűzi.

2. `c(32,87,23,12,1)`

Magyarázat:
Öt számot egy numerikus vektorba fog össze.

3. `c("alma","dinnye","eper","meggy","cseresznye")`

Magyarázat:
Szövegekből karaktervektort hoz létre. Az idézőjelek jelzik, hogy ezek szövegek.

4. `c(T,F,F,T,T,F)`

Magyarázat:
Logikai vektort készít TRUE/FALSE értékekből. T és F a TRUE és FALSE rövid alakjai.

5. `c(5+6,7*4,2^10,3-134)`

Magyarázat:
Előbb kiszámolja a négy kifejezést, majd az eredményeket vektorba teszi: 11, 28, 1024, -131.

6. `c(4<6,6==6,7!=7,7<9)`

Magyarázat:
A feltételeket kiértékeli, majd logikai vektort készít: TRUE, TRUE, FALSE, TRUE.

7. `x=c("András",F,5)`

Magyarázat:
Vegyes típusokat próbál egy vektorba tenni. Mivel van köztük szöveg, R közös típusra alakítja az elemeket: mindegyik karakter lesz ("András", "FALSE", "5"). Fontos: egy atomi vektor elemei azonos típusúak.

8. `c(21,32,43,54,T,F)`

Magyarázat:
Számokat és logikai értékeket kever. R numerikus típusra alakít: TRUE -> 1, FALSE -> 0.

9. `c(1,2,3,4,5,6,7)`

Magyarázat:
Hét egymást követő számot tartalmazó numerikus vektort készít.

2. Sorozatok a : operátorral

```
# start:stop
```
A kettőspont egyszerű, 1-es lépésközű sorozatokat készít a kezdő- és végérték között.

10. `1:30`

Magyarázat:
1-től 30-ig készít egész számokból sorozatot, 1-es lépésközzel.

11. `43:57`
Magyarázat:
43-tól 57-ig növekvő sorozatot készít.

12. `100:50`
Magyarázat:
100-tól 50-ig csökkenő sorozatot készít, -1-es lépésközzel.

13. `10:-10`
Magyarázat:
10-től -10-ig csökkenő sorozatot készít.

14. `-4:4`
Magyarázat:
-4-től 4-ig növekvő sorozatot készít.

15. `1:1000`
Magyarázat:
1-től 1000-ig tartó vektort készít.

16. `1.5:10.5`
Magyarázat:
1.5-től indul, és 1-es lépésekben halad 10.5-ig: 1.5, 2.5, ..., 10.5.

17. `1.5:10`
Magyarázat:
1.5-től indul 1-es lépésekben; 10-et nem tud pontosan elérni, ezért 9.5 az utolsó érték.

18. `1.326:12.985`
Magyarázat:
1.326-tól indul 1-es lépésekben, és addig halad, amíg nem lépi túl a felső határt. A : lépésköze itt is 1.

3. Sorozatok a seq() függvénnyel

19. `seq()`
Magyarázat:
Alapértelmezett argumentumokkal egyetlen elemet ad: 1. A seq() rugalmas sorozatkészítő függvény.

20. `seq(to=40)`
Magyarázat:
Az alapértelmezett 1-es kezdőértéktől 40-ig készít sorozatot.

21. `seq(to=-20)`
Magyarázat:
1-től -20-ig csökkenő sorozatot készít; R automatikusan -1-es irányt választ.

22. `seq(from=20,to=30)`
Magyarázat:
20-tól 30-ig készít sorozatot. Ugyanaz az eredmény, mint a 20:30.

```
# ugyanaz, mint a 20:30
```
A from a kezdőérték, a to a végérték.

23. `seq(from=20,to=30,by=2)`

Magyarázat:
20-tól 30-ig halad 2-es lépésközzel: 20, 22, ..., 30.

24. `seq(from=10,to=101,by=3)`

Magyarázat:
10-től indul 3-as lépésekkel, és a 101-et meg nem haladó utolsó értékig halad.

25. `seq(from=100,to=50)`

Magyarázat:
100-tól 50-ig csökkenő sorozatot készít, automatikus -1-es lépéssel.

26. `seq(from=100,to=50,by=1)`

Magyarázat:
Hibát ad: lefelé kellene haladni, de a by=1 pozitív lépést ír elő. A lépés előjelének összhangban kell lennie az iránnyal.

27. `seq(from=100,to=50,by=-1)`

Magyarázat:
100-tól 50-ig csökken -1-es lépéssel.

28. `seq(from=100,to=50,by=-2)`

Magyarázat:
100-tól 50-ig csökken 2-esével: 100, 98, ..., 50.

29. `seq(from=1.5,to=99,by=2.4)`

Magyarázat:
1.5-től indul, 2.4-es lépésekkel halad, és nem megy 99 fölé.

30. `seq(to=40,by=2,from=10)`

Magyarázat:
10-től 40-ig készít sorozatot 2-es lépésközzel. A névvel megadott argumentumok sorrendje nem számít.

31. `seq(40,2,10)`

Magyarázat:
Pozicionális argumentumok: from=40, to=2, by=10. Hibát ad, mert csökkenni kellene, de a lépés pozitív.

32. `seq(10,40,2)`

Magyarázat:
Pozicionálisan: from=10, to=40, by=2; tehát 10, 12, ..., 40.

33. `seq(from=10,to=50,length.out=9)`

Magyarázat:
Pontosan 9, egyenletesen elosztott értéket készít 10 és 50 között. Itt a lépésközt R számolja ki.

34. `seq(from=10,to=50,by=5)`

Magyarázat:
10-től 50-ig halad 5-ösével: 10, 15, ..., 50.

35. `seq(from=10,to=50,by=5,length.out=9)`

Magyarázat:
Hibát ad: a from, to, by és length.out együtt túl sok/egymást túlszabályozó argumentum. Vagy by-t, vagy length.out-ot érdemes megadni a két végpont mellett.

4. Ismétlés a rep() függvénnyel

36. `rep(x=7,times=10)`

Magyarázat:
A 7-et tízszer ismétli.

37. `rep(x=1,times=100)`

Magyarázat:
Az 1-et százszor ismétli.

38. `rep("András",10)`

Magyarázat:
Az "András" szöveget tízszer ismétli. A második pozicionális argumentum itt a times.

39. `rep(T,3)`

Magyarázat:
A TRUE értéket háromszor ismétli.

40. `1:10`

Magyarázat:
1-től 10-ig tartó vektort készít; ezt a következő rep() példák alapjaként használjuk.

41. `rep(1:10,4)`

Magyarázat:
Az egész 1:10 vektort négyszer ismétli egymás után.

42. `rep(1:4,times=c(1,3,2,5))`

Magyarázat:
Elemenként eltérő ismétlésszámot ad: 1 egyszer, 2 háromszor, 3 kétszer, 4 ötször jelenik meg.

43. `rep(1:4,6)`

Magyarázat:
Az egész 1,2,3,4 mintát hatszor ismétli.

44. `c(rep(1,6),rep(2,6),rep(3,6),rep(4,6))`

Magyarázat:
Hat 1-est, hat 2-est, hat 3-ast és hat 4-est összefűz egyetlen vektorba.

45. `c(1:5,1:6,1:7)`

Magyarázat:
Három sorozatot fűz össze: előbb 1:5, utána 1:6, majd 1:7.

```
# (1,2,3,4,5,1,2,3,4,5,6,1,2,3,4,5,6,7)
```
A komment megmutatja az előző kifejezés szerkezetét.

46. `rep(1:4,times=6)`
Magyarázat:
A teljes 1:4 vektort ismétli meg hatszor.

47. `rep(1:4,each=6)`
Magyarázat:
Minden egyes elemet hatszor ismétel, mielőtt továbblép a következőre.

48. `rep(1,times=10);rep(1,each=10)`
Magyarázat:
Két kifejezés egy sorban, pontosvesszővel elválasztva. Mivel az x csak egyetlen elem, a times=10 és each=10 ugyanazt a tíz darab 1-est adja.

5. Összefoglaló függvények vektorokon

49. `pelda=c(53,21,98,45,33,76,44,4,96,73)`
Magyarázat:
Létrehoz egy 10 elemű numerikus vektort pelda néven. A következő függvények ezen dolgoznak.

50. `mean(pelda)`
Magyarázat:
Kiszámítja a számtani átlagot. Eredmény: 54.3.

51. `median(pelda)`
Magyarázat:
Kiszámítja a mediánt, vagyis rendezés után a középső értéket/két középső átlagát. Eredmény: 49.

52. `sd(pelda)`
Magyarázat:
Kiszámítja a minta szórását. Eredmény kb. 31.1913.

53. `var(pelda)`
Magyarázat:
Kiszámítja a minta varianciáját. Eredmény: 972.9.

54. `sd(pelda)^2`
Magyarázat:
A szórás négyzetét számítja; ez (kerekítési eltéréstől eltekintve) a varianciával egyezik.

55. `min(pelda)`
Magyarázat:
Megadja a legkisebb elemet. Eredmény: 4.

56. `max(pelda)`
Magyarázat:
Megadja a legnagyobb elemet. Eredmény: 98.

57. `length(pelda)`
Magyarázat:
Megadja a vektor elemszámát. Eredmény: 10.

58. `length(123:965)`
Magyarázat:
Megadja, hány egész szám van 123-tól 965-ig, a két végponttal együtt. Eredmény: 843.

59. `sum(pelda)`
Magyarázat:
Összeadja a vektor összes elemét. Eredmény: 543.

60. `sum(pelda)/length(pelda)`
Magyarázat:
Az összeget elosztja az elemszámmal, tehát kézzel kiszámolja az átlagot; ugyanazt kapjuk, mint a mean(pelda).

```
# átlagszámolás másképpen
```
Ez a komment arra hívja fel a figyelmet, hogy az átlag = összeg / elemszám.

61. `prod(pelda)`
Magyarázat:
Összeszorozza a vektor összes elemét.

62. `range(pelda)`
Magyarázat:
Két értéket ad vissza: a minimumot és maximumot. Itt: 4 és 98.

6. Vektorműveletek és újrahasznosítás (recycling)

```
# Vektorműveletek
```
A legtöbb alapművelet R-ben elemenként dolgozik egy vektoron.

63. `vektor1=pelda`
Magyarázat:
A pelda vektor értékét a vektor1 nevű változóhoz rendeli.

64. `vektor2=c("alma","korte","meggy","cseresznye","barack","szilva")`
Magyarázat:
Hat szövegből karaktervektort készít vektor2 néven.

65. `vektor3=c(T,F,F,T,T,F)`
Magyarázat:
Hat logikai értékből logikai vektort készít.

66. `vektor1^2`
Magyarázat:
A vektor1 minden elemét külön-külön négyzetre emeli.

67. `sqrt(vektor1)`
Magyarázat:
A vektor1 minden elemének külön kiszámítja a négyzetgyökét.

68. `nchar(vektor2)`

Magyarázat:
A karaktervektor minden elemének megszámolja a karaktereit; az eredmény numerikus vektor.

69. `vektor1`

Magyarázat:
Kiírja/megjeleníti a vektor1 aktuális tartalmát.

70. `vektor1+40`

Magyarázat:
A vektor1 minden egyes eleméhez 40-et ad.

71. `vektor1*2`

Magyarázat:
A vektor1 minden elemét megszorozza 2-vel.

72. `vektor1/12`

Magyarázat:
A vektor1 minden elemét elosztja 12-vel.

73. `c(1,2,3,4)+c(9,6,3,0)`

Magyarázat:
Azonos hosszúságú vektorokat elemenként ad össze: 1+9, 2+6, 3+3, 4+0.

74. `c(1,2,3,4)*c(9,6,3,0)`

Magyarázat:
Azonos hosszúságú vektorokat elemenként szoroz össze. Ez nem skalárszorzat, hanem elemenkénti szorzás.

75. `c(1,2,3,4,5,6,7,8)+c(1,2)`

Magyarázat:
A rövidebb vektort R újrahasznosítja: 1,2,1,2,1,2,1,2; ezt adja hozzá a hosszabb vektorhoz.

```
# (1,2,3,4,5,6,7,8)
# (1,2,1,2,1,2,1,2)
```
A komment szemlélteti a recyclingot: a rövidebb vektor ismétlődik.

76. `c(1,2,3,4,5,6,7,8,9)+c(3,2,5)`

Magyarázat:
A 3 elemű vektor háromszor ismétlődik, mert a 9 többszöröse a 3-nak. Nincs részleges recycling.

```
# (1,2,3,4,5,6,7,8,9)
# (3,2,5,3,2,5,3,2,5)
```
A rövidebb vektor teljes mintája pontosan kitölti a hosszabbat.

77. `c(1,2,3,4,5,6,7,8,9)+c(3,2)`

Magyarázat:
A 2 elemű vektort ismétli, de 9 nem osztható 2-vel, ezért az utolsó ismétlés félbemarad. R eredményt ad, de figyelmeztetést is: a hosszabb hossz nem többszöröse a rövidebbnek.

```
# (1,2,3,4,5,6,7,8,9)
# (3,2,3,2,3,2,3,2,3)
```
A komment megmutatja a részleges recyclingot; ez tipikusan olyan helyzet, amit érdemes kerülni.

7. Logikai műveletek vektorokon

78. `c(T,T,F) & c(F,T,F)`

Magyarázat:
Elemenkénti logikai ÉS. Eredmény: FALSE, TRUE, FALSE.

79. `c(T,T,F) && c(F,T,F)`

Magyarázat:
A && skaláris/rövidzáras ÉS, nem több elemű vektorok elemenkénti összehasonlítására való. Modern R-ben több elemű operandusokkal hibát ad. Vektorokra & kell.

80. `c(T,T,F) | c(F,T,F)`

Magyarázat:
Elemenkénti logikai VAGY. Eredmény: TRUE, TRUE, FALSE.

81. `c(T,T,F) || c(F,T,F)`

Magyarázat:
A || skaláris/rövidzáras VAGY. Modern R-ben több elemű vektorokkal hibát ad. Vektorokra | kell.

82. `vektor1<50`

Magyarázat:
A vektor1 minden elemét külön összehasonlítja 50-nel; logikai vektort ad arról, mely elemek kisebbek 50-nél.

83. `sample(1:100,10,replace=T)`

Magyarázat:
Véletlenszerűen kiválaszt 10 értéket 1 és 100 közül, visszatevéssel. Emiatt ugyanaz a szám többször is előfordulhat; az eredmény futásonként változhat.

84. `vektor1==50`

Magyarázat:
Elemenként megvizsgálja, mely elemek pontosan 50-ek. A megadott vektorban egyik sem, ezért minden elem FALSE.

85. `vektor1==33`

Magyarázat:
Elemenként megvizsgálja, mely elem egyenlő 33-mal; az ötödik elemnél TRUE.

86. `vektor1!=33`

Magyarázat:
Elemenként azt vizsgálja, mely elemek nem egyenlők 33-mal; mindenhol TRUE, kivéve a 33 helyén.

87. `vektor1<c(46,99,34,65,10,43,96,64,32,83)`

Magyarázat:
Két azonos hosszúságú vektort elemenként hasonlít össze: az első elemet az elsővel, a másodikat a másodikkal stb.

88. `vektor2 < "dinnye"`
Magyarázat:
A vektor2 minden szövegét külön összehasonlítja a "dinnye" szóval rendezési sorrend szerint. A pontos TRUE/FALSE eredmény függhet a nyelvi (locale) beállítástól.

8. Beépített konstansok és speciális értékek

89. `pi`
Magyarázat:
A pi beépített numerikus konstansát adja (kb. 3.14159).

90. `letters`
Magyarázat:
A 26 kis angol betűt tartalmazó beépített karaktervektor: a, b, ..., z.

91. `LETTERS`
Magyarázat:
A 26 nagy angol betűt tartalmazó beépített karaktervektor: A, B, ..., Z.

92. `NA;NaN;Inf`
Magyarázat:
Három speciális értéket mutat: NA = hiányzó/ismeretlen adat; NaN = nem értelmezhető számérték (Not a Number); Inf = pozitív végtelen. A pontosvessző több kifejezést választ el egy sorban.

93. `1/0`
Magyarázat:
Pozitív végtelent ad: Inf.

94. `log(0)`
Magyarázat:
Negatív végtelent ad: -Inf, mert a logaritmus 0-hoz jobbról közelítve -végtelenhez tart.

95. `Inf-Inf`
Magyarázat:
NaN lesz: a végtelen mínusz végtelen nem meghatározott numerikus művelet.

96. `0/0`
Magyarázat:
NaN lesz: 0/0 meghatározatlan.

9. Hiányzó értékek (NA) kezelése

97. `ures=c(1,2,3,4,NA,6,7,8)`
Magyarázat:
Létrehoz egy vektort, amelyben egy érték hiányzik (NA).

98. `mean(ures)`
Magyarázat:
Alapértelmezés szerint az NA miatt az átlag eredménye is NA.

99. `sd(ures)`

Magyarázat:
Alapértelmezés szerint az NA miatt a szórás eredménye is NA.

100. `max(ures)`

Magyarázat:
Alapértelmezés szerint az NA miatt a maximum eredménye is NA.

101. `mean(ures,na.rm=F)`

Magyarázat:
A na.rm=FALSE azt jelenti, hogy ne távolítsa el a hiányzó értéket; ezért az eredmény NA.

102. `mean(ures,na.rm=T)`

Magyarázat:
A na.rm=TRUE kihagyja az NA-t, és a megmaradt értékek átlagát számolja.

103. `sd(ures,na.rm=T)`

Magyarázat:
Kihagyja az NA-t, és a megmaradt értékekből számolja a minta szórását.